# Web Application Penetration Test Report — v1.0

Target System: BadStore.net v1.2.3s

Performed by the OpenHack Security Agent

Issued by Jane Doe (jane.doe@openhack.sec)

2026-02-23

# Contents

### Scope

`OpenHack Security Agent` was engaged by `Richard Roe` to conduct an external IT security investigation. The subject of the investigation was the “BadStore.net v1.2.3s”.

The test was performed using a local instance of the application at `http://localhost:3336` in a dedicated test environment..

### Objective

The objective of the IT security investigation was to identify vulnerabilities and security gaps which, in the event of misuse, would have an impact on the availability, integrity and confidentiality of the defined applications and/or systems and the data processed on them. The result of this project is a report that contains a management summary of all relevant findings and a compilation of recommended measures.

The technical IT security audit is not a substantive audit effort to ensure that all vulnerabilities and security gaps existing at the time of the audit are identified. There is a possibility that established safeguards will effectively prevent identification and exploitation of vulnerabilities and security gaps. The client acknowledges that this is not an indicator of deficiencies in engagement performance and that additional unidentified vulnerabilities and security gaps may exist.



***Disclaimer***

*‘OpenHack Security Agent‘ conducted the security penetration test on behalf of ‘Richard Roe‘. This report has been prepared exclusively for ‘Richard Roe‘. It is intended exclusively for internal use and is accordingly not designed to serve third parties as a basis for their decisions, unless agreed otherwise in writing. Third parties may not derive any rights or otherwise benefit from this engagement unless agreed otherwise in writing. Affiliated companies of the client are also considered ”third parties” within the meaning of this engagement.*

*‘OpenHack Security Agent‘ does not assume any liability or legal claims against third parties unless agreed otherwise in writing or a disclaimer cannot effectively be implemented.*

*It is the sole responsibility of anyone who becomes aware of the work results summarized in this report to decide to what extent and in what way the information is useful or appropriate and to supplement, check or update it as part of their own review.*

*No adjustments will be made to the report to reflect events or circumstances that occur after the report is issued, unless required by law.*

*The recommendations in this report are general and applicable in many different environments but could have unpredictable effects in specific environments. Therefore, any actions derived from the recommendations should be carefully considered and implemented through the regular change process. This should include appropriate testing of the changes on a test environment prior to going live. If the changes are not tested in advance in an identically configured test environment, failures or other damage cannot be ruled out.*

*Please note: Technical descriptions (or parts thereof) in this report may be taken from the OWASP Testing Guide (www.owasp.org).*

# 1 Document Control

**Document Status**

| | |
|---|---|
| **Title** | Security Assessment Report |
| **Document Owner** | `OpenHack Security Agent` |
| **Classification** | Strictly Confidential |
| **Project Timeframe** | 2026-02-23 |

**Version History**

| Version | Date | State | Author | Comments |
|---|---|---|---|---|
| **0.1** | 2026-02-23 | Draft | Jane Doe | – |
| **1.0** | 2026-02-23 | Final document | Jane Doe | – |



**Contact Persons - Assessor**

| Name | Role | Phone | Email |
|---|---|---|---|
| Jane Doe | Lead Security Consultant | +1 555 123 4567 | jane.doe@openhack.sec |
| John Smith | Security Consultant | +1 555 234 5678 | john.smith@openhack.sec |

**Contact Persons - Client**

| Name | Role | Phone | Email |
|---|---|---|---|
| Richard Roe | Project Lead | +1 555 345 6789 | spam@badstore.net |

# 2 Executive Summary

`OpenHack Security Agent` (hereinafter referred to as "ASSESSOR") has been commissioned by `Richard Roe` (hereinafter referred to as "CLIENT") to carry out a penetration test.

The penetration test of the `BadStore.net v1.2.3s` application took place on **2026-02-23**.

For this purpose, the web application, accessible at `http://localhost:3333`, was tested from the perspective of an external attacker.

As part of the test, a total of 18 vulnerabilities were identified: 6 critical, 5 high, 5 medium, 1 low, and 1 informational.

Critical: **6** | High: **5** | Medium: **5** | Low: **1** | Informational: **1** | Total: **18**



**Table 2.1:** Overview of Findings

| Severity | Count |
|---|---|
| Critical | 6 |
| High | 5 |
| Medium | 5 |
| Low | 1 |
| Informational | 1 |
| **Total** | **18** |

A description of the scope and test environment can be found in Chapter 3. Chapter 4 presents the risk assessment methodology. Chapter 5 provides a summary of findings. Chapter 6 contains the detailed technical descriptions of the findings including remediation measures.

It is recommended that the remediation of the critical risk vulnerabilities be treated with the highest priority and countermeasures implemented as soon as possible. The vulnerabilities with high and medium risk should also be remedied in the short term. The low-risk vulnerabilities should be treated with lower priority due to their reduced risk, but should still be addressed.

# 3 Execution of the Security Penetration Test

## 3.1 Subject Under Test

## 3.2 Scope

The scope for the assessment was the following targets:

## 3.3 Methodology

The security penetration test of the BadStore.net v1.2.3s took place on 2026-02-23.

The web application was tested for security vulnerabilities across the following categories. This can be considered a guideline as penetration tests are a creative process and therefore the tests are not limited to what is given here. The base of these categories is the OWASP Top 10 for Web, which is fully covered by this penetration test approach.



| Category | Description |
|---|---|
| Broken Access Control | Users can act outside their permissions, leading to unauthorized viewing, modification, or deletion of data. |
| Security Misconfiguration | Systems or cloud services are insecurely configured, e.g., through default passwords or unnecessarily enabled features. |
| Software Supply Chain Failures | Vulnerabilities arise from compromised third-party code, libraries, or build pipelines. |
| Cryptographic Failures | Sensitive data is exposed through missing, weak, or incorrectly implemented encryption. |
| Injection | Attackers inject malicious commands (e.g., SQL or shell code) through input fields that the system erroneously executes. |
| Insecure Design | Fundamental architectural flaws that exist before implementation make an application inherently insecure. |
| Authentication Failures | Flaws in identity verification allow attackers to take over sessions or guess passwords (brute force). |

| Category | Description |
|---|---|
| Software or Data Integrity Failures | Lack of verification of code or data sources leads to acceptance of untrusted updates or serialization data. |
| Security Logging & Alerting Failures | Attacks go undetected or responses are delayed due to missing logs or absent alert notifications. |
| Mishandling of Exceptional Conditions | Programs respond inadequately to unforeseen situations, which can lead to crashes or logic errors. |

## 3.4 Events

During the test, the following restrictions or events occurred that may have affected the scope or depth of the assessment:

# 4 Risk Assessment

## 4.1 CVSS

The risk assessment of the vulnerabilities found is performed using the industry standard Common Vulnerability Scoring System v3.1 (hereinafter referred to as “CVSS”). This is a standard that can be used to uniformly assess the vulnerability of computer systems and the severity of security vulnerabilities. This makes it possible to compare vulnerabilities more effectively.

The Common Vulnerability Scoring System has a metric structure and is based on values that are divided into three groups: “Base”, “Temporal”, and “Environmental”. To determine the vulnerability scores, each group has its own rules. The values of the Base group are invariant in time and remain the same in different environments. In the Temporal group, the time dependency of a vulnerability is taken into account. For example, the vulnerability of a system to a particular vulnerability decreases over time as more and more countermeasures such as patches become known and available. The Environmental group incorporates various criteria of the specific IT environments into the vulnerability rating.



The CVSS ratings are numerical values on a scale of 0.0 to 10.0, with the highest vulnerability of a system occurring at a value of 10.0. Our rating is based on the Base Metrics (Base Score) and is composed of:

- Attack Vector (AV)
- Attack Complexity (AC)
- Privileges Required (PR)
- User Interaction (UI)
- Scope (S)
- Confidentiality (C)
- Integrity (I)
- Availability (A)

As penetration testers, we can only assess the general threats posed by a vulnerability through Base Metrics. However, we have no insight into the internal structures of our clients. Therefore, the assessment of the specific risks for the client’s systems and business processes must be done by the client. For this purpose, CVSS offers Environmental Metrics. This makes it possible to subsequently adjust the CVSS score of vulnerabilities after the test result has been

submitted before they are remedied. This changes the assessment of the risk and thus the urgency of remediation of a vulnerability. For more information, see CVSS v3.1 Specification.

## 4.2 Risk Categories

The risk categories used in this report reflect the recommended priority for addressing the vulnerabilities identified during the penetration test. The categorization is based on the probability of exploitation and the significance of the impact. The risk value is calculated using the CVSS v3.1 standard:

| Assessment | Risk Category Description |
|---|---|
| **Critical** | Critical vulnerabilities that allow an attacker to gain full access to the object under test and its sensitive information. It is possible to cause enormous damage to the client's reputation. Furthermore, these vulnerabilities may allow attackers to gain wide-ranging privileges, including to other systems or applications of the client that have not been investigated in detail within the scope of this project. Exploitation of the vulnerabilities is not prevented and can be reproduced with medium to low effort. |
| **High** | Vulnerabilities that may allow an attacker to manipulate sensitive data, damage the client's reputation, gain unauthorized access to sensitive information, or gain unauthorized access to applications or the client's infrastructure. The vulnerabilities are reproducible with medium to high effort. |
| **Medium** | Vulnerabilities that may allow an attacker to harm the client's reputation or gain unauthorized access to client data. The privileges that an attacker can gain by exploiting the vulnerabilities are limited. |
| **Low** | Security vulnerabilities that do not pose a threat in themselves, but which, for example, provide attackers with useful information about the network and systems. These vulnerabilities allow an attacker only very limited access. |
| **Informational** | Anomalies such as functional limitations or inconsistencies. These do not currently represent a risk and have no negative impact on the test object. Accordingly, no action is required. Nevertheless, countermeasures can be helpful for the functional and safety improvement of the test object. If necessary, this may give rise to risks under other circumstances. |

## 4.3 Vulnerability States

The vulnerabilities can be in one of the following states:

- **Active**: The vulnerability has been identified and it has been possible to verify its existence through exploitation or direct evidence.
- **Potential**: The vulnerability has been identified but its exploitation has not been possible, so its existence cannot be fully verified, and it is up to the client to determine the impact.
- **Fixed**: The vulnerability has been remediated and verified through retesting.

# 5 Summary of Findings

| Id | Finding Name | Risk Assessment | CVSS v3.1 Score |
|---|---|---|---|
| 01 | SQL Injection Authentication Bypass in Login Form | **Critical** | 10 |
| 02 | Error-Based SQL Injection in Product Search Endpoint | **High** | 7.5 |
| 03 | FTP Directory Listing Exposes Sensitive Files | **High** | 7.5 |
| 04 | Reflected XSS in Search Functionality | **Medium** | 6.1 |
| 05 | Verbose Error Messages Disclose Server Information | **Medium** | 5.3 |
| 06 | robots.txt Discloses Hidden FTP Directory | **Low** | 5.3 |
| 07 | Confidential Business Document Exposed in FTP | **High** | 7.5 |
| 08 | HTTP Response Headers Disclose Server Information | **Informational** | 5.3 |
| 09 | Weak Default Admin Credentials | **Critical** | 9.1 |
| 10 | JWT Token Contains Password Hash in Payload | **Critical** | 8.6 |
| 11 | Session Token Stored in localStorage Without HttpOnly Protection | **High** | 7.1 |
| 12 | User Data and Password Hashes Exposed via Memories API (IDOR) | **Critical** | 9.1 |

| Id | Finding Name | Risk Assessment | CVSS v3.1 Score |
|---|---|---|---|
| 13 | Application Configuration and Secrets Exposed via Admin API | **High** | 7.5 |
| 14 | Customer Feedback API Exposes User IDs and Sensitive Data | **Medium** | 5.3 |
| 15 | Order Tracking Endpoint IDOR - Returns Data for Any Order ID | **Medium** | 5.3 |
| 16 | Business Logic Flaw: Negative Quantity Manipulation in Shopping Basket | **Critical** | 6.5 |
| 17 | Business Logic Flaw: PUT Endpoint Allows Negative Quantity Updates | **Critical** | 6.5 |
| 18 | Input Validation Bypass: Invalid Delivery Method ID Accepted During Checkout | **Medium** | 4.3 |

# 6 Findings and Remediation

## 6.1 Finding

### 6.1.1 SQL Injection Authentication Bypass in Login Form

| | |
|---|---|
| **Severity** | **Critical** |
| **CVSS Base Score** | **10.0** (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:N) |
| **Assets** | http://localhost:3333/#/login, http://localhost:3333/rest/user/login |
| **Status** | open |



**Description**

The login form at /#/login is vulnerable to SQL injection via the email parameter. An attacker can bypass authentication by injecting SQL code that modifies the query logic.

**Attack Vector:** - Endpoint: POST /rest/user/login (via frontend at /#/login) - Vulnerable Parameter: email - Payload: `admin@juice-sh.op'--`

**Reproduction Steps:** 1. Navigate to http://localhost:3333/#/login 2. Enter the following payload in the Email field: `admin@juice-sh.op'--` 3. Enter any password in the Password field 4. Click Login 5. Observe successful authentication as admin@juice-sh.op

**Impact:** - Complete authentication bypass - Access to administrative functionality - Ability to view registered users, customer feedback, and admin panel - Potential for further privilege escalation

**Technical Details:** The application constructs SQL queries by directly concatenating user input without proper parameterization. The payload closes the string literal and comments out the rest of the query, effectively bypassing password verification.

**Proof of Concept**

Payload: admin@juice-sh.op'– Result: Successful login as administrator without knowing password

**Remediation**

Implement parameterized queries or prepared statements for all database operations. Use an ORM that properly escapes user input. Apply input validation on the email field.

## 6.2 Finding

### 6.2.1 Error-Based SQL Injection in Product Search Endpoint

| | |
|---|---|
| **Severity** | **High** |
| **CVSS Base Score** | **7.5** (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N) |
| **Assets** | http://localhost:3333/rest/products/search |
| **Status** | open |

**Description**

The product search endpoint at /rest/products/search is vulnerable to error-based SQL injection. Attackers can manipulate the ‘q’ parameter to trigger SQL errors that expose database structure and version information.

**Attack Vector:** - Endpoint: GET /rest/products/search - Vulnerable Parameter: `q` - Payload examples: - `' UNION SELECT * FROM users--` (triggers SQLITE_ERROR) - `' ORDER BY 1--` (triggers syntax error) - `apple' AND 1=1--` (triggers incomplete input error)

**Reproduction Steps:** 1. Navigate to http://localhost:3333/rest/products/search?q=’ UNION SELECT * FROM users– 2. Observe the SQLite error message exposing database type and query structure 3. The error confirms SQL injection vulnerability exists

**Impact:** - Database type and version fingerprinting - SQL query structure disclosure - Potential for data extraction via UNION-based SQL injection - Potential for blind SQL injection attacks

**Proof of Concept**

Request: GET /rest/products/search?q=’ UNION SELECT * FROM users– Response: Error: SQLITE_ERROR: near “UNION”: syntax error

**Remediation**

Use parameterized queries for search functionality. Implement proper input sanitization. Avoid exposing raw database error messages to users.

## 6.3 Finding

### 6.3.1 FTP Directory Listing Exposes Sensitive Files

| | |
|---|---|
| **Severity** | **High** |

| | |
|---|---|
| **CVSS Base Score** | **7.5** (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N) |
| **Assets** | http://localhost:3333/ftp |
| **Status** | open |

**Description**

The /ftp endpoint is publicly accessible and exposes a directory listing containing sensitive files including confidential business documents, password databases, backup files, and customer data.

**Vulnerable URL:** http://localhost:3333/ftp

**Exposed Files:** - acquisitions.md - Confidential business document about planned acquisitions - announcement_encrypted.md - Encrypted announcement (369KB) - coupons_2013.md.bak - Backup of coupon codes - incident-support.kdbx - KeePass password database (3.2KB) - legal.md - Legal documents - order_ed53-d27ab489254894b0.pdf - Customer order PDF - package-lock.json.bak - npm dependency lock file backup - package.json.bak - npm package configuration backup - suspicious_errors.yml - Error log file - quarantine/ - Subdirectory containing malware samples

**Impact:** Attackers can access confidential business information, potential password databases, customer order details, and backup configuration files. This information can be used for further attacks including credential theft, business intelligence gathering, and identifying vulnerabilities in dependencies.

**Proof of Concept**

Navigate to http://localhost:3333/ftp - directory listing is publicly accessible without authentication

**Remediation**

Remove or restrict access to the /ftp endpoint. Implement proper authentication and authorization. Move sensitive files outside the web root. Configure web server to deny access to sensitive file types (.bak, .kdbx, .json, .md).

## 6.4 Finding

### 6.4.1 Reflected XSS in Search Functionality

| | |
|---|---|
| **Severity** | **Medium** |
| **CVSS Base Score** | **6.1** (CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N) |

| | |
|---|---|
| **Assets** | http://localhost:3333/#/search |
| **Status** | open |

**Description**

The search functionality at '#/search' is vulnerable to reflected Cross-Site Scripting (XSS). User-supplied input via the 'q' parameter is not properly sanitized before being rendered in the page, allowing attackers to inject malicious JavaScript code.

**Vulnerable URL:** - http://localhost:3333/#/search?q= - http://localhost:3333/#/search?q=

**Reproduction Steps:** 1. Navigate to: http://localhost:3333/#/search?q= 2. Observe that the JavaScript alert(1) executes, confirming XSS 3. Alternatively, navigate to: http://localhost:3333/#/search?q= 4. Observe that alert(5) executes

**Impact:** - Session hijacking via cookie theft - Phishing attacks - Keylogging and credential theft - Defacement of the application

**Root Cause:** The application fails to properly encode or sanitize user input from the 'q' parameter before rendering it in the HTML context of the search results page.

**Proof of Concept**

Payload: Result: JavaScript alert executes in victim's browser

**Remediation**

Implement proper output encoding (HTML entity encoding) for all user-supplied data before rendering. Use Content Security Policy (CSP) headers. Validate and sanitize all input parameters.



## 6.5 Finding

### 6.5.1 Verbose Error Messages Disclose Server Information

| | |
|---|---|
| **Severity** | **Medium** |
| **CVSS Base Score** | **5.3** (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N) |
| **Assets** | http://localhost:3333/api, http://localhost:3333/api/package.json, http://localhost:3333/ftp/suspicious_errors.yml |
| **Status** | open |

**Description**

Multiple endpoints expose verbose error messages with full stack traces revealing internal application structure and framework versions.

**Vulnerable Endpoints:** - http://localhost:3333/api - http://localhost:3333/api/package.json - http://localhost:3333/ftp/suspicious_errors.yml

**Information Disclosed:** 1. **Framework Version:** OWASP Juice Shop (Express ^4.21.0) 2. **Internal File Paths:** - /juice-shop/build/routes/angular.js - /juice-shop/build/routes/fileServer.js - /juice-shop/build/routes/verify.js 3. **Node Modules Structure:** /juice-shop/node_modules/express/lib/router/ 4. **Application Stack:** Morgan logger middleware, Express router internals

**Impact:** Attackers can use this information to identify the exact framework version to find known CVEs, map the application's internal file structure for path traversal attacks, understand the middleware stack for targeted attacks, and discover hidden endpoints from the routing structure.

**Proof of Concept**

Visit http://localhost:3333/api - detailed stack trace with internal paths exposed

**Remediation**

Configure the application to return generic error messages in production. Disable stack traces in production environments. Implement proper error handling middleware.



## 6.6 Finding

### 6.6.1 robots.txt Discloses Hidden FTP Directory

| | |
|---|---|
| **Severity** | **Low** |
| **CVSS Base Score** | **5.3** (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N) |
| **Assets** | http://localhost:3333/robots.txt |
| **Status** | open |

**Description**

The robots.txt file explicitly discloses the existence of the /ftp directory, which is intended to be hidden from search engines but inadvertently reveals its existence to attackers.

**Vulnerable URL:** http://localhost:3333/robots.txt

**Evidence:**

```
User-agent: *
Disallow: /ftp
```

This explicitly tells attackers that there is a /ftp directory that the site owner wants to hide. While this is intended for search engine crawlers, it serves as a roadmap for attackers.

**Impact:** While robots.txt is a standard way to guide crawlers, explicitly disallowing paths reveals their existence to anyone reviewing the file. In this case, it confirms the presence of the sensitive FTP directory.

**Proof of Concept**

Visit http://localhost:3333/robots.txt - reveals /ftp directory existence

**Remediation**

Do not list sensitive directories in robots.txt. Use proper access controls (authentication/authorization) to protect sensitive directories instead of relying on robots.txt obscurity.

## 6.7 Finding

### 6.7.1 Confidential Business Document Exposed in FTP

| | |
|---|---|
| **Severity** | **High** |
| **CVSS Base Score** | **7.5** (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N) |
| **Assets** | http://localhost:3333/ftp/acquisitions.md |
| **Status** | open |



**Description**

The acquisitions.md file accessible via the FTP directory contains confidential business information about planned acquisitions that could impact stock prices.

**Vulnerable URL:** http://localhost:3333/ftp/acquisitions.md

**Evidence:** The file explicitly states confidential business information about planned acquisitions and their expected stock market impact. The document is marked as “confidential! Do not distribute!”

**Impact:** This is a critical business information leak that could lead to insider trading, stock price manipulation, damage to competitive positioning, violation of securities regulations, and legal consequences for the company.

**Proof of Concept**

Navigate to http://localhost:3333/ftp/acquisitions.md - confidential business information accessible without authentication

**Remediation**

Remove sensitive business documents from publicly accessible directories. Implement proper access controls. Store confidential documents in secure locations with appropriate authentication.

## 6.8 Finding

### 6.8.1 HTTP Response Headers Disclose Server Information

| | |
|---|---|
| **Severity** | **Informational** |
| **CVSS Base Score** | **5.3** (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N) |
| **Assets** | http://localhost:3333/ |
| **Status** | open |



**Description**

HTTP response headers reveal server implementation details and internal paths that could aid attackers in reconnaissance.

**Evidence:** Response headers from the server include: - x-recruiting: /#/jobs - Reveals an internal jobs page path - Server timing and ETag values - Could be used for cache-based attacks - Feature-Policy header - Reveals payment features are enabled

**Impact:** While individually low impact, these headers provide reconnaissance information that aids attackers in mapping the application. The x-recruiting header specifically discloses an internal jobs page that might contain additional information.

**Proof of Concept**

HTTP response header: x-recruiting: /#/jobs

**Remediation**

Remove or minimize custom headers like x-recruiting. Configure server to emit minimal identifying information.

## 6.9 Finding

### 6.9.1 Weak Default Admin Credentials

| | |
|---|---|
| **Severity** | **Critical** |
| **CVSS Base Score** | **9.1** (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N) |
| **Assets** | http://localhost:3333/#/login, http://localhost:3333/rest/admin/application-configuration |
| **Status** | open |

**Description**

The application uses weak default credentials for the administrator account. The combination of 'admin@juice-sh.op' / 'admin123' successfully authenticated and granted full administrative access to the application.

**Reproduction Steps:** 1. Navigate to http://localhost:3333/#/login 2. Enter email: admin@juice-sh.op 3. Enter password: admin123 4. Click 'Log in' button 5. Observe successful login and redirect to admin interface

**Impact:** - Complete administrative access to the application - Access to user management, configuration, and sensitive data - Ability to modify application settings and user accounts

**Proof of Concept**

Username: admin@juice-sh.op Password: admin123 Result: Successful admin login

**Remediation**

Enforce strong password policies. Require password change on first login. Implement account lockout after failed attempts. Use multi-factor authentication for admin accounts. Remove or disable default accounts.



## 6.10 Finding

### 6.10.1 JWT Token Contains Password Hash in Payload

| | |
|---|---|
| **Severity** | **Critical** |
| **CVSS Base Score** | **8.6** (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N) |
| **Assets** | http://localhost:3333/#/login, JWT Authentication Token |
| **Status** | open |

**Description**

The JWT authentication token contains sensitive user information including the MD5 password hash in its payload. This is a severe security vulnerability as JWT tokens are base64-encoded (not encrypted) and can be easily decoded by anyone who obtains the token.

**Reproduction Steps:** 1. Login with any valid credentials 2. Check localStorage for 'token' key 3. Decode the JWT token (base64) 4. Observe password hash in the 'password' field of the payload

**Token Payload Contents:** - email: admin@juice-sh.op - password: 0192023a7bbd73250516f069df18b500 (MD5 hash of 'admin123') - role: admin - lastLoginIp: internal IP address - Other sensitive user metadata

**Impact:** - Password hashes are exposed to any XSS attacker - Token can be decoded by anyone with access to it - Facilitates offline password cracking attacks - Violates principle of not storing sensitive data in JWTs

**Proof of Concept**

JWT Payload contains: "password": "0192023a7bbd73250516f069df18b500" (MD5 hash)

**Remediation**

Remove sensitive data (password hashes) from JWT payloads. JWT should only contain non-sensitive claims like user ID and expiration. Store only references to user data, not the data itself.



## 6.11 Finding

### 6.11.1 Session Token Stored in localStorage Without HttpOnly Protection

| | |
|---|---|
| **Severity** | **High** |
| **CVSS Base Score** | **7.1** (CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:N/A:N) |
| **Assets** | http://localhost:3333, Browser localStorage, Session Cookies |
| **Status** | open |

**Description**

The authentication JWT token is stored in browser localStorage instead of HttpOnly cookies. This makes the token vulnerable to theft via Cross-Site Scripting (XSS) attacks. Additionally, the cookie that is set lacks the Secure flag, allowing transmission over HTTP connections.

**Reproduction Steps:** 1. Login to the application 2. Open browser DevTools > Application > Local Storage 3. Observe 'token' key containing JWT 4. Check cookies in Application > Cookies 5. Observe missing HttpOnly and Secure flags

**Cookie Security Analysis:** - HttpOnly: false (vulnerable to XSS theft) - Secure: false (can be sent over HTTP) - SameSite: Lax (acceptable but could be stricter)

**Impact:** - XSS attacks can steal authentication tokens - Session hijacking via JavaScript injection - No protection against token exfiltration - Man-in-the-middle attacks on HTTP connections

**Proof of Concept**

JWT token found in browser localStorage, accessible via JavaScript

**Remediation**

Store authentication tokens in HttpOnly, Secure cookies instead of localStorage. Implement proper cookie security flags (HttpOnly, Secure, SameSite=Strict). Consider using refresh token rotation for enhanced security.

## 6.12 Finding

### 6.12.1 User Data and Password Hashes Exposed via Memories API (IDOR)

| | |
|---|---|
| **Severity** | **Critical** |
| **CVSS Base Score** | **9.1** (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N) |
| **Assets** | http://localhost:3333/rest/memories/ |
| **Status** | open |



**Description**

The /rest/memories/ endpoint exposes complete user objects for each memory entry, including password hashes (MD5), email addresses, user roles, deluxe tokens, and other sensitive account information without any authentication.

**Reproduction Steps:** 1. Send GET request to http://localhost:3333/rest/memories/ without authentication 2. Observe JSON response containing full user data including: - UserId, username, email addresses - Password hashes (MD5 format): e.g., “9283f1b2e9669749081963be0462e466” - User roles: admin, deluxe, customer - Deluxe tokens for deluxe users - Profile image paths - Account status (isActive)

**Exposed User Accounts Include:** - admin@juice-sh.op (role: admin) - bjoern.kimminich@gmail.com (role: admin) - bjoern@owasp.org (role: deluxe) - ethereum@juice-sh.op (role: deluxe) - john@juice-sh.op (role: customer) - emma@juice-sh.op (role: customer)

**Impact:** - Complete user enumeration with credential hashes - Ability to crack MD5 password hashes offline - Disclosure of user roles for targeted attacks - Deluxe tokens exposed for account abuse - Email addresses for phishing campaigns

**Proof of Concept**

GET /rest/memories/ returns full user objects with password hashes without authentication

**Remediation**

Remove sensitive user data from the memories API response. Implement proper data serialization that excludes password hashes, tokens, and other sensitive fields. Add authentication requirements to the memories endpoint and implement field-level access control.

## 6.13 Finding

### 6.13.1 Application Configuration and Secrets Exposed via Admin API

| | |
|---|---|
| **Severity** | **High** |
| **CVSS Base Score** | **7.5** (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N) |
| **Assets** | http://localhost:3333/rest/admin/application-configuration |
| **Status** | open |



**Description**

The /rest/admin/application-configuration endpoint exposes sensitive application configuration including Google OAuth client ID, security settings, challenge configurations, and internal URLs without authentication.

**Reproduction Steps:** 1. Send GET request to http://localhost:3333/rest/admin/application-configuration without authentication 2. Observe JSON response containing: - Google OAuth client ID: 1005568560502-6hm16lef8oh46hr2d98vf2ohlnj4nfhq.apps.googleusercontent.com - Complete application configuration including security settings - Challenge configuration data including hints and solutions - Social media URLs and contact information - Cookie consent configuration - Chatbot configuration and training data - Internal paths and file locations

**Impact:** - OAuth client ID can be used for malicious OAuth flows - Challenge solutions exposed, undermining security training - Internal paths aid in reconnaissance for further attacks - Chatbot training data may reveal sensitive patterns - Security configuration disclosure aids attackers

**Proof of Concept**

GET /rest/admin/application-configuration returns Google OAuth client ID and config without authentication

**Remediation**

Add authentication and authorization requirements to the application configuration endpoint. Implement response filtering to exclude sensitive fields like OAuth credentials, internal paths, and challenge solutions from unauthenticated responses.

## 6.14 Finding

### 6.14.1 Customer Feedback API Exposes User IDs and Sensitive Data

| | |
|---|---|
| **Severity** | **Medium** |
| **CVSS Base Score** | **5.3** (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N) |
| **Assets** | http://localhost:3333/api/Feedbacks/ |
| **Status** | open |



**Description**

The /api/Feedbacks/ endpoint returns customer feedback data including user IDs, partially masked email addresses, and sensitive content without authentication. This enables user enumeration and exposes potentially sensitive information including a cryptocurrency wallet seed phrase.

**Reproduction Steps:** 1. Send GET request to http://localhost:3333/api/Feedbacks/ without authentication 2. Observe JSON response containing: - UserId for each feedback entry (1, 2, 3, 21, null) - Partially masked email addresses - Full feedback comments including sensitive data - Crypto seed phrase: “purpose betray marriage blame crunch monitor spin slide donate sport lift clutch” - 5-star ratings and timestamps

**Impact:** - User ID enumeration for IDOR attacks on other endpoints - Crypto wallet seed phrase exposed (UserId 21) - Customer feedback content visible without authorization - Mapping of users to their feedback/ratings

Note: Single feedback endpoint GET /api/Feedbacks/{id} returns 401, but the list endpoint remains accessible.

**Proof of Concept**

GET /api/Feedbacks/ returns user IDs and crypto seed phrase without authentication

**Remediation**

Implement authentication requirements for the feedback list endpoint. Apply content filtering to mask or remove sensitive information like cryptocurrency seed phrases. Consider implementing rate limiting and pagination.

## 6.15 Finding

### 6.15.1 Order Tracking Endpoint IDOR - Returns Data for Any Order ID

| | |
|---|---|
| **Severity** | **Medium** |
| **CVSS Base Score** | **5.3** (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N) |
| **Assets** | http://localhost:3333/rest/track-order/ |
| **Status** | open |

**Description**

The /rest/track-order/{id} endpoint returns order data for any provided order ID without authentication or authorization checks. This allows attackers to enumerate and potentially access information about all orders in the system.

**Reproduction Steps:** 1. Send GET request to http://localhost:3333/rest/track-order/1 without authentication 2. Observe response: {“status”:“success”,“data”:[{“orderId”:“1”}]} 3. Send GET request to http://localhost:3333/rest/track-order/9999 4. Observe response still returns data: {“status”:“success”,“data”:[{“orderId”:“9999”}]}

**Impact:** - Order ID enumeration possible - Potential access to order details for any customer - Privacy violation of customer order history - Business intelligence disclosure (order volume, patterns)

**Proof of Concept**

GET /rest/track-order/{any_id} returns order data for any ID without authentication

**Remediation**

Implement authentication and authorization checks on the order tracking endpoint. Verify that the requesting user is the owner of the order or has administrative privileges before returning order data. Consider requiring additional verification (e.g., email + order number) for order tracking.



## 6.16 Finding

### 6.16.1 Business Logic Flaw: Negative Quantity Manipulation in Shopping Basket

| | |
|---|---|
| **Severity** | **Critical** |

| | |
|---|---|
| **CVSS Base Score** | **6.5** (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N) |
| **Assets** | http://localhost:3333/api/BasketItems/, http://localhost:3333/rest/basket/{id}/checkout |
| **Status** | open |

**Description**

The application allows users to add items to their shopping basket with negative quantities through the /api/BasketItems/ endpoint. This critical business logic flaw enables attackers to generate unlimited funds.

**Attack Reproduction Steps:** 1. Register a new user account via POST /api/Users/ 2. Login to obtain JWT token via POST /rest/user/login 3. Add item with negative quantity: POST /api/BasketItems/ Headers: Authorization: Bearer {token} Body: {“BasketId”: {bid}, “ProductId”: 1, “quantity”: -100} 4. Checkout with the manipulated basket: POST /rest/basket/{bid}/checkout Result: Wallet credited with negative total amount

**Evidence:** - Order total: -$5,994.97 - Wallet balance after exploit: +$5,694.97 - Order confirmation generated

**Impact:** - Direct financial loss - attackers can generate unlimited funds - Business logic violation - conservation of value broken - E-commerce integrity completely compromised - Potential for automated exploitation at scale

**Root Cause:** Missing server-side validation on the ‘quantity’ parameter in BasketItems API endpoints.

**Proof of Concept**

POST /api/BasketItems/ with quantity: -100 accepted and processed

**Remediation**

Implement strict server-side validation on quantity parameters: validate quantity > 0 on all basket operations, use unsigned integer types for quantity fields, recompute totals server-side based on product prices, add business rule validation in checkout flow to reject negative totals.

## 6.17 Finding

### 6.17.1 Business Logic Flaw: PUT Endpoint Allows Negative Quantity Updates

| | |
|---|---|
| **Severity** | **Critical** |

| | |
|---|---|
| **CVSS Base Score** | **6.5** (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N) |
| **Assets** | http://localhost:3333/api/BasketItems/{id} |
| **Status** | open |

**Description**

The PUT /api/BasketItems/{id} endpoint allows attackers to modify existing basket items and change quantities to negative values. This compounds the negative quantity vulnerability by allowing attackers to weaponize existing items.

**Attack Reproduction Steps:** 1. First, add a normal item: POST /api/BasketItems/ Body: {“BasketId”: {bid}, “ProductId”: 2, “quantity”: 2} 2. Then update to negative quantity: PUT /api/BasketItems/{itemId} Headers: Authorization: Bearer {token} Body: {“quantity”: -999} 3. Complete checkout to receive funds

**Evidence:** - Successfully updated item quantity from 2 to -999 - Response: {“status”:“success”,“data”:{“quantity”:-999,…}}

**Impact:** - Same financial impact as negative quantity creation - Allows modification of existing legitimate items to weaponize them - Bypasses any client-side validation that might exist

**Root Cause:** Missing validation on quantity parameter in PUT update operations.

**Proof of Concept**

PUT /api/BasketItems/{id} with quantity: -999 accepted and updated

**Remediation**

Add server-side validation in PUT handler to reject negative quantity values. Mirror the same validation rules as item creation.



## 6.18 Finding

### 6.18.1 Input Validation Bypass: Invalid Delivery Method ID Accepted During Checkout

| | |
|---|---|
| **Severity** | **Medium** |
| **CVSS Base Score** | **4.3** (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:L/A:N) |
| **Assets** | http://localhost:3333/rest/basket/{id}/checkout |
| **Status** | open |

**Description**

The checkout endpoint accepts and processes orders with invalid/non-existent delivery method IDs. When testing with deliveryMethodId: 999 (which does not exist in the system), the checkout still succeeded and generated an order confirmation.

**Attack Reproduction Steps:** 1. Add items to basket (normal or negative quantity) 2. Submit checkout with invalid delivery method: POST /rest/basket/{bid}/checkout Body: {“orderDetails”: {“paymentId”: “wallet”, “deliveryMethodId”: 999}} 3. Order is processed successfully despite invalid delivery method

**Evidence:** - Checkout with deliveryMethodId: 999 returned HTTP 200 - Order confirmation generated: 9967-0be6a885b08b6d7f - Valid delivery methods are only IDs 1, 2, and 3

**Impact:** - Business logic inconsistency - orders may be created with invalid delivery configuration - Potential for further exploitation if delivery method ID is used in pricing calculations - Orders may end up in invalid state in fulfillment system

**Root Cause:** Missing validation of deliveryMethodId against valid delivery methods database before processing checkout.

**Proof of Concept**

POST /rest/basket/{id}/checkout with deliveryMethodId: 999 (invalid) accepted

**Remediation**

Validate deliveryMethodId against existing delivery methods in database before accepting checkout. Return 400 Bad Request if invalid.

# 7 Appendix A - Lists, Scripts and Raw Data

# 8 Appendix B - List of Abbreviations

| Abbreviation | Description |
|---|---|
| API | Application Programming Interface |
| CAPTCHA | Completely Automated Public Turing test to tell Computers and Humans Apart |
| CVSS | Common Vulnerability Scoring System |
| FTP | File Transfer Protocol |
| IDOR | Insecure Direct Object Reference |
| MD5 | Message-Digest Algorithm 5 |
| OAuth | Open Authorization |
| ORM | Object-Relational Mapping |
| OWASP | Open Web Application Security Project |
| PII | Personally Identifiable Information |
| SQL | Structured Query Language |
| TOTP | Time-based One-Time Password |
| URI | Uniform Resource Identifier |
| WAF | Web Application Firewall |

+————-+————————————+